Companion to Bird & de Moor, Algebra of Programming

§2. Functions and Categories

§2.1. Categories

§2.2. Functors

§2.3. Natural transformations

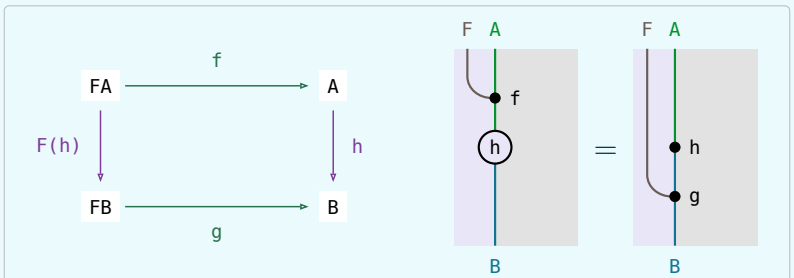
§2.4. Constructing datatypes

§2.5. Products and coproducts

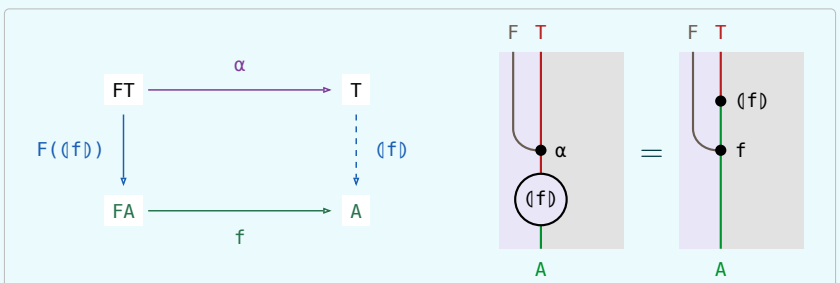
§2.6. Initial algebras

(2.6a)

The **initial algebra** `α : F(T)→T` has exactly one F-homomorphism `(|f|) : T→A` to every F-algebra `f : F(A)→A`.



`(h : f → g) ≜ f h = F(h) g`

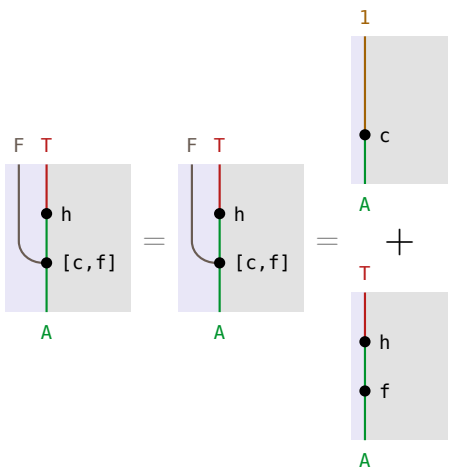


`Map(f) ⇒ α (|f|) = F((|f|)) f`

`α h = F(h) [c,f] ⇔ zero h = c ∧ succ h = h f` (2.6b)

`h` is a homomorphism from `α = [zero,succ]` to `[c,f]` exactly when `h` sends `zero` to `c` and `succ` then `h` equals `h` then `f`

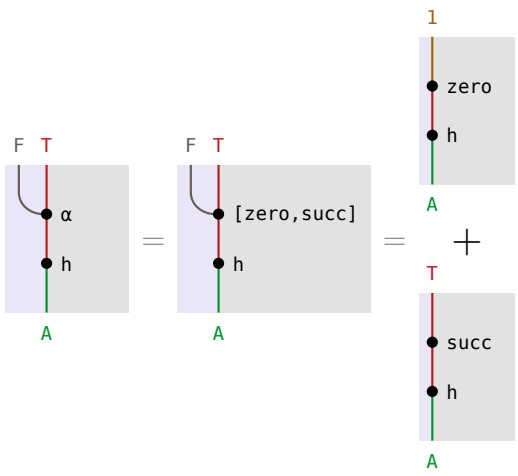
the right side



(1) (2) (3)

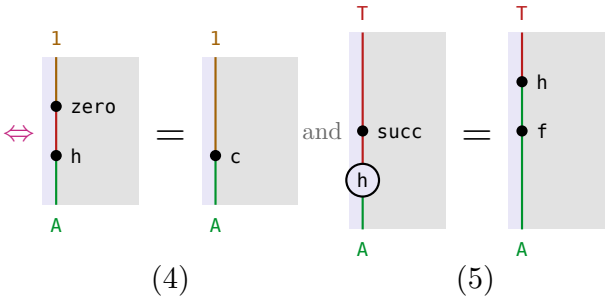
1	`F(h) [c,f]`	definition of `F`
2	`= (𝟙 + h) [c,f]`	coproduct
3	`= [c,h f]`	

the left side



(1) (2) (3)

1	`α h`	since `α = [zero,succ]`
2	`= [zero,succ] h`	coproduct
3	`= [zero h,succ h]`	cancellation, `zero` arm

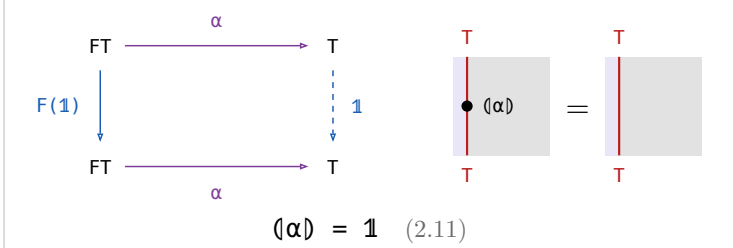


(4) (5)

4	`⇔ [zero h,succ h] = [c,h f] ⇒ zero h = c`	cancellation, `succ` arm
5	and `[zero h,succ h] = [c,h f] ⇒ succ h = h f`	

§2.6.1. Reflection

(2.6.1a)



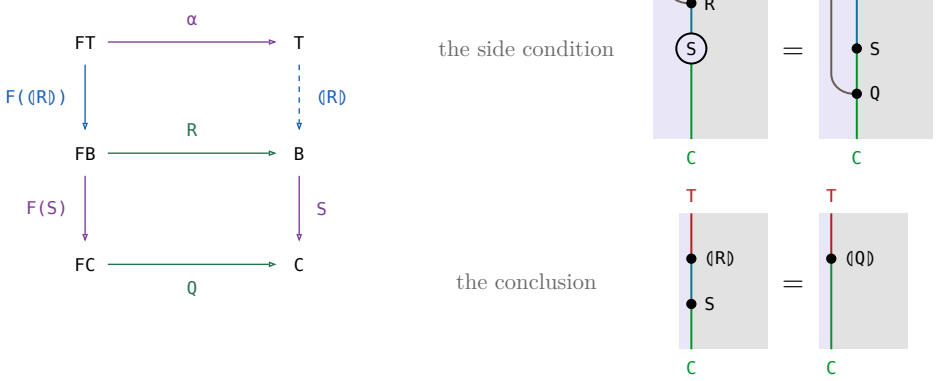
`(α) = 𝟙` (2.11)

Taking a value apart with `α` and putting it straight back is doing nothing.

§2.6.2. `R S = F(S) Q ⇒ (R) S = (Q)` — folding with `R`, then applying `S`, is folding with `Q`

When `S : B→C` is an F-homomorphism from `R : F(B)→B` to `Q : F(C)→C`, folding with `R` and then applying `S` is folding with `Q`.

(2.6.2a)



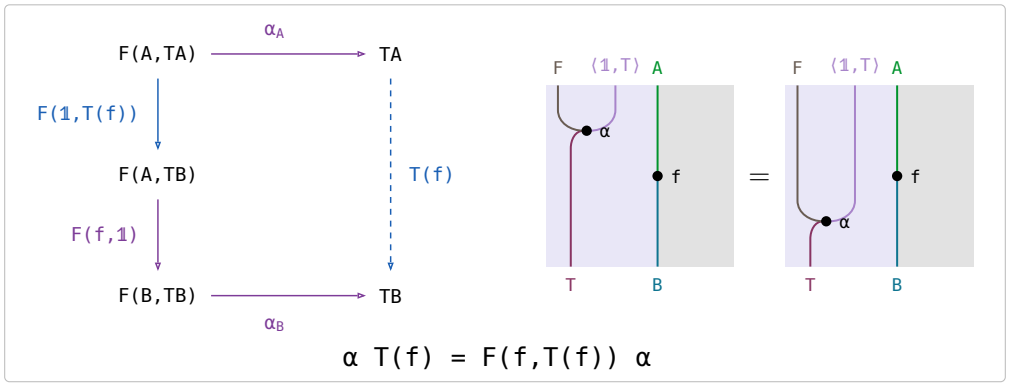

```
R S = F(S) Q ⇒ ⦇R⦈ S = ⦇Q⦈
```
(2.12)

§2.7. Type functors

(2.7a)

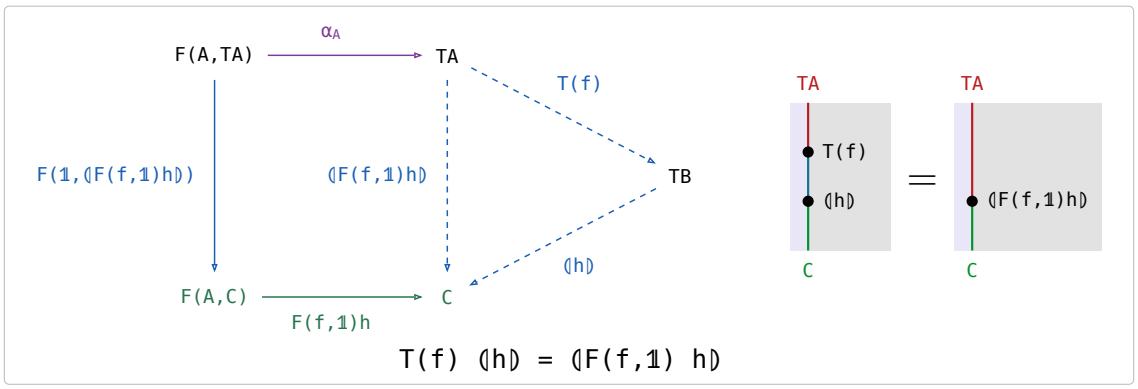
`T(R) ≜ ⦇F(R,𝟙) α⦈` — the type functor `T` acts on a map `f : A→B` through the initial algebras α_A `: F(A,TA)→TA`.

(2.7b)



- `F : 𝒜×𝒜→𝒜` is a bifunctor and a wire is a unary functor, so the two arguments are packed first: `⟨𝟙,T⟩ : 𝒜→𝒜×𝒜` sends `A` to `(A,TA)`, and `F(⟨𝟙,T⟩(A))` is `F(A,TA)`.
- The picture is three wires — `F`, `⟨𝟙,T⟩`, and the object — and the region between the first two is `𝒜×𝒜`.
- `α` is then an ordinary natural transformation `F∘⟨𝟙,T⟩⇒T`: its bead eats the `F` and `⟨𝟙,T⟩` wires, and the `T` wire is born under it.
- `F(f,T(f))` costs no notation. It is the bead `f` on the object wire with `⟨𝟙,T⟩` and `F` running past: `⟨𝟙,T⟩` is what turns `f` into the pair `(f,T(f))`, and `F` is what applies it.
- The law is the naturality of `α`, which is exactly the freedom to slide that `f` bead past it.

(2.7c)



(2.7d)

	name	law	what it says
1	functor	`T(𝟙) = 𝟙` and `T(R) T(S) = T(R S)`	Acting by the identity changes nothing, and two actions in a row are one action.
2	type functor fusion	`T(f) ⦇h⦈ = ⦇F(f,𝟙) h⦈`	A relator action followed by a fold is a single fold — the intermediate structure is never built. The side condition holds because `F` is a bifunctor — `F(R,𝟙)F(𝟙,⦇Q⦈)=F(R,⦇Q⦈)=F(𝟙,⦇Q⦈)F(R,𝟙)`.
3	naturality of `α`	`α T(f) = F(f,T(f)) α`	Building and then mapping is the same as mapping the parts and then building, so `α` is natural from `G(R)=F(R,T(R))` to `T`.